Practical Assignment 4 – A-Frame VR Scene

Student ID: 241482

Name: Amina Smailova

Theme: Underwater Research Base

Motion: Pulse

Interaction: Gaze

Layout: Corridor

E1 – Component Life Cycle

Prediction: I expect init() to run when the component is created. update() should run after initialization and again when a component property is changed. pause() and play() should run when they are called. remove() should run when the component is removed. I expect oldData in update() to contain the previous component data.

Experiment and Result

I tested the x1482-motion component on the Water Sample exhibit. First, I changed the amplitude from 0.10 to 0.25 using setAttribute(). The Console showed that update() was called and oldData contained the previous amplitude value of 0.10. Then I called pause() and play(). The pulse animation stopped and then continued. Finally, I removed the x1482-motion component and then removed the Water Sample entity. The Console showed the REMOVE lifecycle event.

Explanation

The experiment showed that init() runs when the component is created, and update() is also called during initial setup. When a property is changed with setAttribute(), update() runs again and oldData contains the previous component values. pause() and play() control the component activity. remove() runs when the component is removed and can be used to restore or clean up its state.

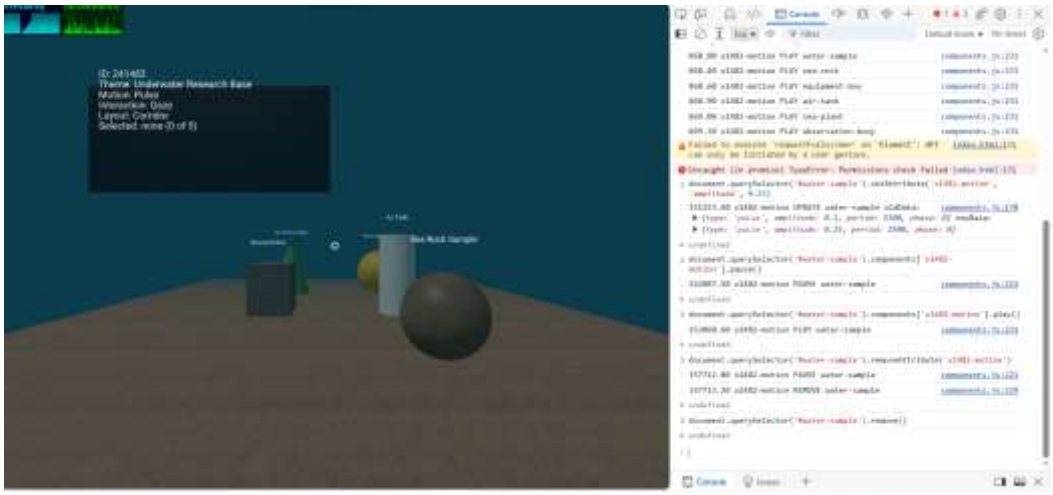


Figure 1. x1482-motion component life cycle test in the browser Console.

E2 - Checking Chapter 8

Prediction

I expect this.data to contain only the schema values of the x1482-texture component. When I change the repeat property using setAttribute(), this.data should update to the new value. Runtime Three.js objects such as texture, material, and mesh should not be stored in the schema.

Experiment and Result

I tested the x1482-texture component on the Sea Rock exhibit. The initial component data contained src, repeat, normalSrc, and anisotropy values. I changed the repeat property using setAttribute() from 2 2 to 5 5. After the change, this.data showed repeat values of 5 5. I also inspected the Three.js mesh and confirmed that it exists as a separate runtime object.

Explanation

The schema should contain simple configuration data, such as texture source, repeat values, and anisotropy. Runtime objects such as texture, material, and mesh should be stored on the component instance (this), not inside this.data, because they are created and changed while the scene is running. <a-assets> is used to preload assets such as images, while AFRAME.registerSystem() is used for shared scene-level logic and communication between components.

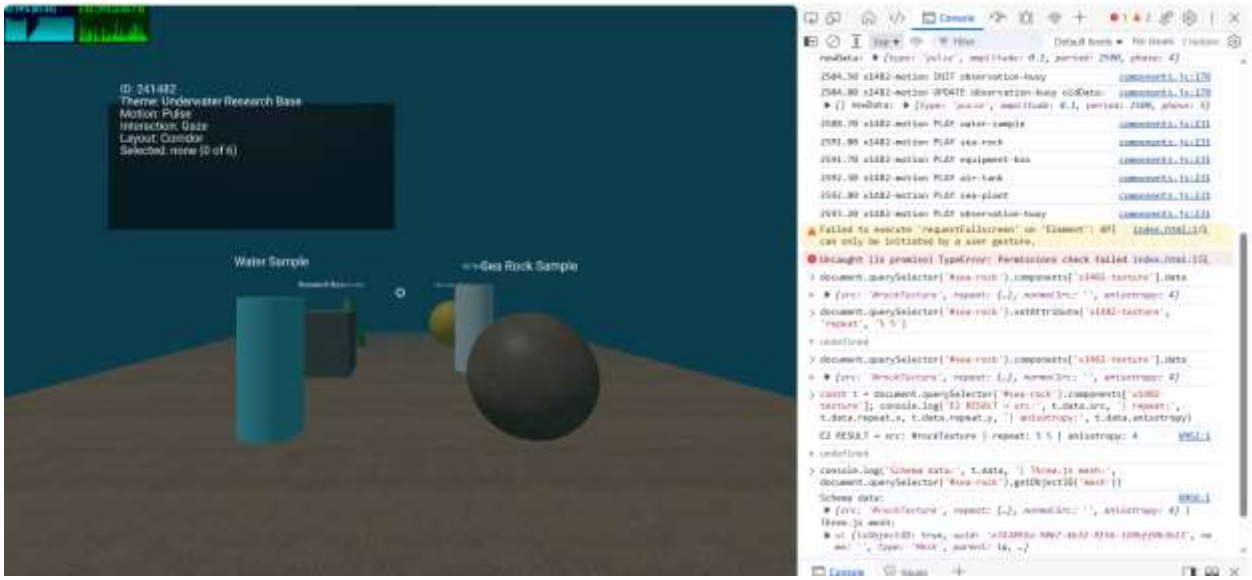


Figure 2. x1482-texture data update and Three.js runtime mesh inspection.

E3 – Texture Filtering

Prediction

I expect texture filtering to affect the appearance of the floor, especially at a distance and at an angle. The default filtering should look acceptable, while mipmaps and anisotropic filtering should make the repeated floor texture clearer and reduce visible distortion.

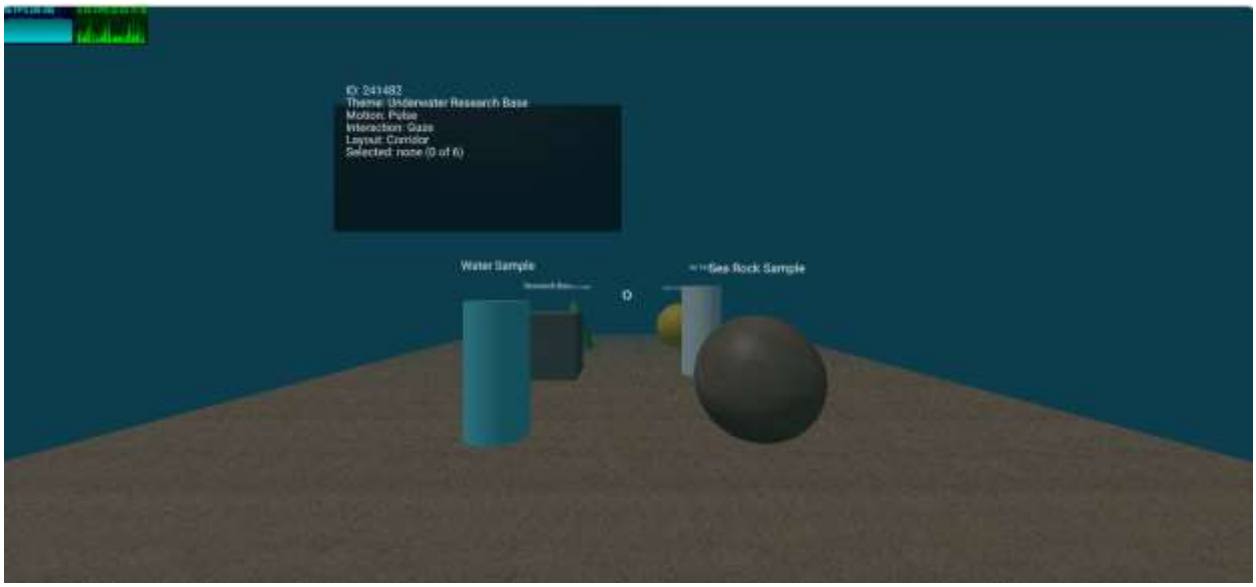


Figure 3. Floor texture with Chapter 8 filtering

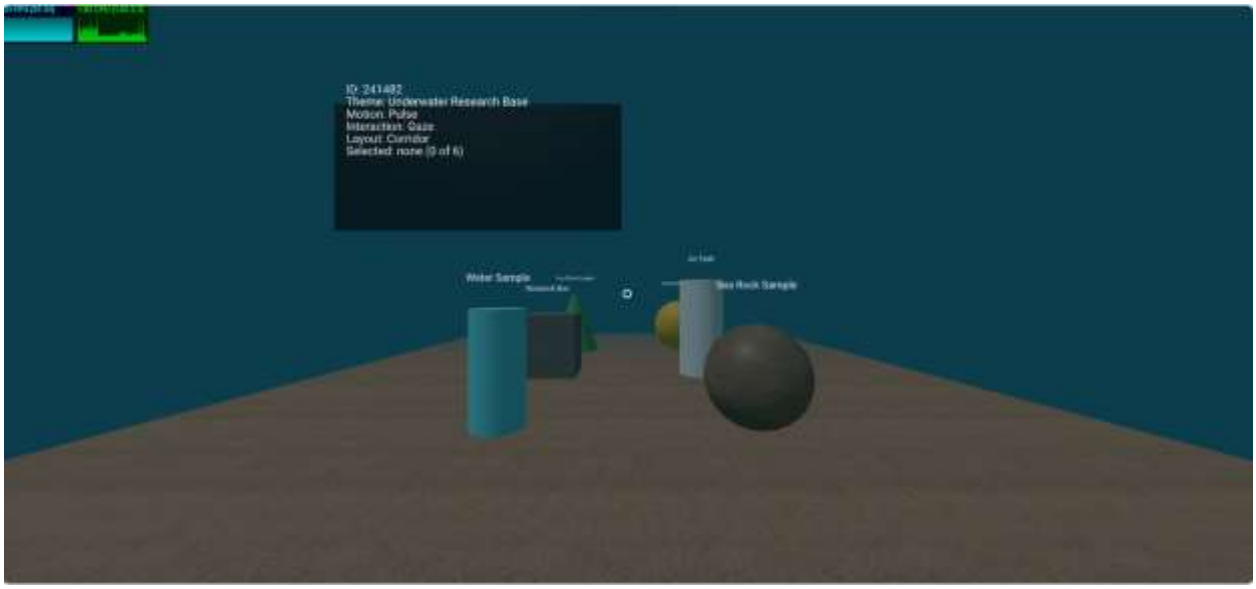


Figure 4. Floor texture with Three.js default filtering.

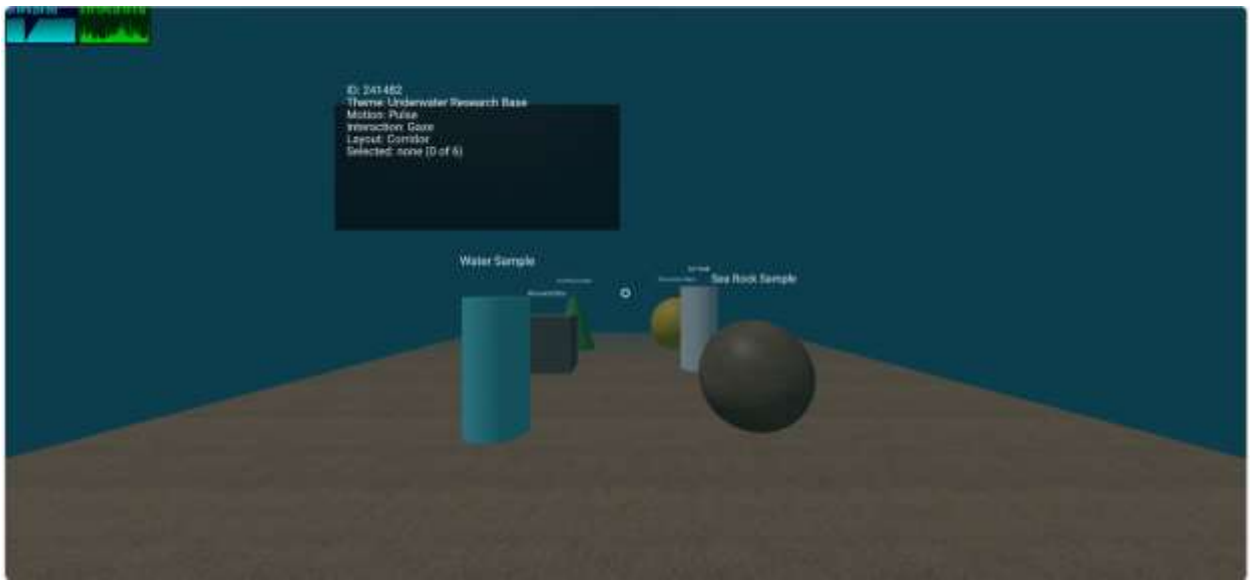


Figure 5. Floor texture with linear filtering, mipmaps, and anisotropic filtering

Experiment and Result

Three texture filtering configurations were tested from the same camera position. The first test used NearestFilter without mipmaps. The second test used the Three.js default linear filtering with mipmaps and anisotropy 1. The third test used linear filtering, mipmaps, and anisotropic filtering. The first configuration looked sharper and more pixelated, while linear filtering produced a smoother floor texture. Anisotropic filtering improved the appearance of the texture when viewed at an angle and at a distance.

Explanation

Minification filtering controls a texture when it appears smaller on the screen, while magnification filtering controls it when it appears larger. Mipmaps provide smaller versions of a texture for distant surfaces and help reduce visual aliasing. Anisotropic filtering improves texture quality on surfaces viewed at steep angles. For the final scene, linear filtering with mipmaps and anisotropy is suitable because the floor extends into the distance.

E4 – What <a-scene> Builds

Prediction

I expect A-Frame to automatically create the Three.js renderer, scene object, and camera. This should require less manual WebXR setup than my previous PA3 project.

Experiment and Result

I inspected the A-Frame scene in the browser Console. sceneEl.renderer returned a WebGLRenderer, sceneEl.object3D returned a Three.js Scene, and sceneEl.camera returned a PerspectiveCamera. In the current desktop session, sceneEl.renderer.xr.enabled returned false because no immersive WebXR session was active.

Explanation

A-Frame automatically creates and manages the renderer, Three.js scene, and camera behind <a-scene>. It also provides its WebXR integration when an immersive session is requested. In PA3, I worked at a lower level and manually configured Three.js and the WebXR session, including renderer.xr.enabled and AR session logic. Raw Three.js is better when detailed control of WebXR features such as hit testing and anchors is required. A-Frame is better for quickly building a declarative VR scene with entities, components, cameras, and interactions.



Figure 6. Inspection of the A-Frame renderer, Three.js scene, camera, and XR state

E5 – Performance

Prediction

I expect the performance to decrease when 60 additional exhibits are added. The number of draw calls and triangles should increase because the scene will contain more geometry

Test	FPS	Draw calls	Triangles
Original scene	60	17	11,516
+60 exhibits	60	77	12,236
After optimization	60	18	12,236

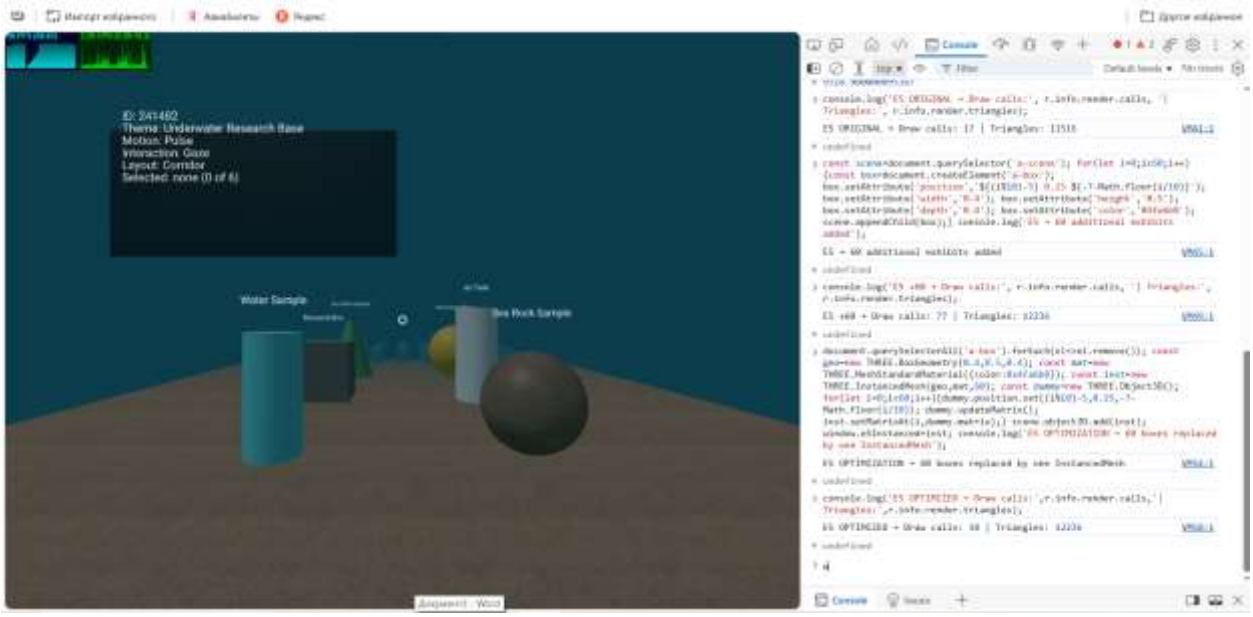


Figure 7. Performance measurements before and after InstancedMesh optimization.

Experiment and Result

The original scene ran at 60 FPS with 17 draw calls and 11,516 triangles. After adding 60 additional box exhibits, the scene still ran at 60 FPS, but draw calls increased to 77 and triangles increased to 12,236. I then replaced the 60 separate boxes with one Three.js InstancedMesh. After optimization, the scene remained at 60 FPS and 12,236 triangles, while draw calls decreased from 77 to 18.

Explanation

Adding many separate objects increased the number of draw calls because each object was rendered separately. InstancedMesh allows multiple copies of the same geometry and material to be rendered together. This reduced the draw calls from 77 to 18 without reducing the number of displayed boxes. In this test, FPS remained at 60, but the lower number of draw calls makes the scene more efficient.

E6 – Comfort and Design

The exhibits are placed in two rows along a corridor, with enough space between them for clear navigation and viewing. The labels are positioned above the exhibits so they can be read without blocking the objects. Text size was chosen to remain readable from the normal viewing distance and to provide a comfortable viewing angle.

The gaze interaction uses a 1.5-second fuse time. This gives the user enough time to focus on an exhibit and reduces accidental selections.

One possible comfort risk is continuous movement with WASD, especially when the scene is viewed in VR. Smooth movement can cause discomfort for some users. A possible mitigation is to reduce movement speed or use teleportation in a future VR version. The simple corridor layout, clear spacing, and fixed information panel also help users understand the scene without unnecessary movement.

Development Log

During development, I found a problem with the exhibit selection and gallery interaction. I tested the gaze selection, information panel, motion controls, and object removal in the browser. After correcting the interaction logic, the six exhibits worked correctly with the M and X controls. The final fix was saved in commit d40541a – “Complete PA4 exhibit interaction and gallery”.